

```
--1. What is the total amount each customer spent at the restaurant
SELECT customer_id, SUM(price) as total_amt
FROM sales s
LEFT JOIN menu m
ON s.product_id = m.product_id
GROUP BY customer_id
ORDER BY total_amt DESC;
```

	customer_id character varying (1) 🔒	total_amt bigint 🔒
1	A	76
2	B	74
3	C	36

```
--2. How many days has each customer visited the restaurant?
with cust_ord as (
SELECT DISTINCT customer_id, order_date
FROM sales
ORDER BY customer_id
)
SELECT customer_id, COUNT(*) as Days_visit_count
FROM cust_ord
GROUP BY customer_id;
```

	customer_id character varying (1) 🔒	days_visit_count bigint 🔒
1	A	4
2	B	6
3	C	2

```
--3. What was the first item from the menu purchased by each customer?
with first_item as (
SELECT DISTINCT customer_id,
FIRST_VALUE(product_id) OVER(
PARTITION BY customer_id
ORDER BY order_date
) as first_product_id
FROM sales
)
SELECT customer_id, first_product_id, product_name
FROM first_item fi
LEFT JOIN menu m
on fi.first_product_id = m.product_id;
```

	customer_id character varying (1) 🔒	first_product_id integer 🔒	product_name character varying (5) 🔒
1	A	1	sushi
2	B	2	curry
3	C	3	ramen

```
--4. What is the most purchased item on the menu and how many
-- times was it purchased by all customers?
with most_purchased as (
SELECT product_id, COUNT(product_id) as count_of_orders
FROM sales
group by product_id
)
select mp.product_id, m.product_name, mp.count_of_orders
from most_purchased mp
left join menu m
on mp.product_id = m.product_id
ORDER BY mp.count_of_orders DESC
LIMIT 1;
```

	product_id integer	product_name character varying (5)	count_of_orders bigint
1	3	ramen	8

```
--5. Which item was the most popular for each customer?
with pop_ord as (
select customer_id, product_id, count(*) as order_count
from sales
group by customer_id, product_id
ORDER BY customer_id,order_count DESC
),
ranked_pop_ord as (
select customer_id,product_id,order_count,DENSE_RANK() OVER(
PARTITION BY customer_id
ORDER BY order_count DESC
) as ranked_prod
from pop_ord
)
select customer_id,product_name
from ranked_pop_ord rpo
left join
menu m
on rpo.product_id = m.product_id
where ranked_prod = 1
order by customer_id;
```

	customer_id character varying (1)	product_name character varying (5)
1	A	ramen
2	B	sushi
3	B	curry
4	B	ramen
5	C	ramen

```
--6. Which item was purchased first by the customer after
-- they became a member?

select DISTINCT s.customer_id, FIRST_VALUE(product_id) OVER(
PARTITION BY s.customer_id
ORDER BY order_date
) as product_id_mbrshp
from sales s
left join members m
on s.customer_id = m.customer_id
where s.order_date >= m.join_date
order by customer_id;
```

	customer_id character varying (1)	product_id_mbrshp integer
1	A	2
2	B	1

```
--7. Which item was purchased just before
-- the customer became a member?
with purchase_bef_mem as (
    select s.customer_id,order_date,product_id,join_date
    from sales s
    left join members m
    on s.customer_id = m.customer_id
    where s.order_date < m.join_date
    order by s.customer_id,order_date DESC
),
final as (
select customer_id,order_date,pbm.product_id,customer_id,join_date,product_name,DENSE_RANK() OVER(
PARTITION BY customer_id
ORDER BY order_date DESC
) as ranked_ord
from purchase_bef_mem pbm
left join menu m
on pbm.product_id = m.product_id
)
select *
from final
where ranked_ord = 1;
```

	customer_id character varying (1) 🔒	order_date date 🔒	product_id integer 🔒	customer_id character varying (1) 🔒	join_date date 🔒	product_name character varying (5) 🔒	ranked_ord bigint 🔒
1	A	2021-01-01	1	A	2021-01-07	sushi	1
2	A	2021-01-01	2	A	2021-01-07	curry	1
3	B	2021-01-04	1	B	2021-01-09	sushi	1

```
--8. What is the total items and amount spent for each
-- member before they became a member?

select s.customer_id,count(*) as total_items, sum(price) as total_amt
from sales s
left join members m
on s.customer_id = m.customer_id
left join menu mu
on s.product_id = mu.product_id
where s.order_date < m.join_date
group by s.customer_id
order by s.customer_id;
```

	customer_id character varying (1) 🔒	total_items bigint 🔒	total_amt bigint 🔒
1	A	2	25
2	B	3	40

```
--9. If each $1 spent equates to 10 points and sushi
-- has a 2x points multiplier - how many points would
-- each customer have?

with sales_menu as (
    select customer_id,s.product_id,product_name,price,
    CASE
        when product_name = 'sushi' then 20
        else 10
    END as multiplier
    from sales s
    left join menu m
    on s.product_id = m.product_id
)
select customer_id,sum(price * multiplier) as total_pts
from sales_menu
group by customer_id
order by total_pts DESC;
```

	customer_id character varying (1) 🔒	total_pts bigint 🔒
1	B	940
2	A	860
3	C	360

```

--10. In the first week after a customer joins the program
-- (including their join date) they earn 2x points on all
-- items, not just sushi - how many points do customer
-- A and B have at the end of January?

with sales_fil as (
  select *
  from sales
  where customer_id in (select distinct customer_id from members)
    and EXTRACT(month from order_date) = 1
)
, sales_mem_fil as (
  select sf.customer_id,product_id
  from sales_fil sf
  left join members m
  on sf.customer_id = m.customer_id
  where order_date between join_date and join_date + INTERVAL '6 days'
)
select customer_id, sum(price*20) as total_pts
from sales_mem_fil as smf
left join menu m
on smf.product_id = m.product_id
group by customer_id
order by total_pts DESC;

```

	customer_id character varying (1) 🔒	total_pts bigint 🔒
1	A	1020
2	B	200

-- Bonus Questions

```

-- Join all the things
with sales_menu as (
  select customer_id,order_date,product_name,price
  from sales s
  left join menu m
  using (product_id)
  order by customer_id
)
select sm.customer_id,order_date,product_name,price,
CASE
  when order_date >= join_date then 'Y'
  ELSE 'N'
END as members
from sales_menu sm
left join members m
on sm.customer_id = m.customer_id
order by customer_id,order_date

```

	customer_id character varying (1) 🔒	order_date date 🔒	product_name character varying (5) 🔒	price integer 🔒	members text 🔒
1	A	2021-01-01	sushi	10	N
2	A	2021-01-01	curry	15	N
3	A	2021-01-07	curry	15	Y
4	A	2021-01-10	ramen	12	Y
5	A	2021-01-11	ramen	12	Y
6	A	2021-01-11	ramen	12	Y
7	B	2021-01-01	curry	15	N
8	B	2021-01-02	curry	15	N
9	B	2021-01-04	sushi	10	N
10	B	2021-01-11	sushi	10	Y
11	B	2021-01-16	ramen	12	Y
12	B	2021-02-01	ramen	12	Y
13	C	2021-01-01	ramen	12	N
14	C	2021-01-01	ramen	12	N
15	C	2021-01-07	ramen	12	N

```
-- Rank ALL The Things

with sales_menu as (
select customer_id,order_date,product_name,price
from sales s
left join menu m
using (product_id)
order by customer_id
),
fin as (
select sm.customer_id,order_date,product_name,price,
CASE
    when order_date >= join_date then 'Y'
    ELSE 'N'
END as members
from sales_menu sm
left join members m
on sm.customer_id = m.customer_id
order by customer_id,order_date
)
select *, -- The thing to remember in this query was that Dense rank
CASE -- function runs first but the case statement only decides what to show
    WHEN members = 'N' then NULL
    ELSE DENSE_RANK() OVER (
        PARTITION BY customer_id, members
        ORDER BY order_date
    ) END as ranking
from fin;
```

	customer_id character varying (1) 🔒	order_date date 🔒	product_name character varying (5) 🔒	price integer 🔒	members text 🔒	ranking bigint 🔒
1	A	2021-01-01	sushi	10	N	[null]
2	A	2021-01-01	curry	15	N	[null]
3	A	2021-01-07	curry	15	Y	1
4	A	2021-01-10	ramen	12	Y	2
5	A	2021-01-11	ramen	12	Y	3
6	A	2021-01-11	ramen	12	Y	3
7	B	2021-01-01	curry	15	N	[null]
8	B	2021-01-02	curry	15	N	[null]
9	B	2021-01-04	sushi	10	N	[null]
10	B	2021-01-11	sushi	10	Y	1
11	B	2021-01-16	ramen	12	Y	2
12	B	2021-02-01	ramen	12	Y	3
13	C	2021-01-01	ramen	12	N	[null]
14	C	2021-01-01	ramen	12	N	[null]
15	C	2021-01-07	ramen	12	N	[null]